

CloudNotes Service Troubleshooting Lab

Complete Evidence Report - All Three Scenarios

Task 1: Service Startup Failure - Configuration Error

Problem Description

The CloudNotes service fails to start due to a configuration-related error. The application exits immediately during startup because a required configuration key is missing.

Error Evidence

```
$ /c/Python314/python ./app/server.py
CRITICAL: Configuration error: Missing required
setting 'database_file_path'
```

Root Cause Analysis

The application configuration file uses the wrong key name. The server code expects `database_file_path` but the JSON configuration provides `db_file_path`.

✗ Before (Wrong)

```
{
  "host": "127.0.0.1",
  "port": 8080,
  "log_file_path":
"logs/application.log",
  "db_file_path":
```

✓ After (Correct)

```
{
  "host": "127.0.0.1",
  "port": 8080,
  "log_file_path":
"logs/application.log",
  "database_file_path":
```

```
"data/notes.json"  
}
```

```
"data/notes.json"  
}
```

Fix Applied

Command:

```
sed -i 's/"db_file_path"/"database_file_path"/g' app/config.json
```

Verification

✓ Configuration now correct:

The application progresses past configuration parsing and continues to initialize.

Task 2: Permission and Ownership Failure

Problem Description

After fixing the configuration, the application encounters a file permission error. The application cannot write to the log file, preventing proper initialization and logging.

Error Evidence

```
CRITICAL: Permission denied when opening log file at
...\logs/application.log.
Run 'ls -l' to check ownership and permissions.
```

Permission Inspection

BEFORE FIX:

```
$ ls -l logs/application.log
-r--r--r-- 1 NallapureddyRishitha 197121 0 Jul 9 09:29
logs/application.log
```

Analysis:

- File mode: **444** (r--r--r--)
- Owner: read only (no write permission)
- Group: read only
- Others: read only
- **Issue:** No write permission for anyone - application cannot log

Root Cause

The incident simulation script intentionally set restrictive permissions (chmod 400) on the log file, simulating a deployment misconfiguration. This prevents the daemon process from writing log entries.

Fix Applied

Command:

```
chmod 644 logs/application.log
```

Rationale: This sets read+write for owner, read for group and others, following the principle of least privilege. NOT using chmod 777 as that would be overly permissive.

Permission Verification**AFTER FIX:**

```
$ ls -l logs/application.log
-rw-r--r-- 1 NallapureddyRishitha 197121 0 Jul 9 09:29
logs/application.log
```

Result:

- File mode: **644** (rw-r--r--)
- ✓ Owner can now READ and WRITE
- ✓ Group and others can READ
- ✓ Application can now write log entries

Task 3: Port Conflict and Service Recovery

Problem Description

The CloudNotes service requires port 8080 to bind and serve HTTP requests. Another process (rogue service) is already listening on this port, preventing the main application from starting.

Scenario Setup

STEP 1: Rogue Service Running

```
$ /c/Python314/python ./scripts/rogue_service.py &  
[1] 891  
Rogue service started with PID: 891
```

STEP 2: Port Conflict Detected

```
$ netstat -ano | grep 8080  
TCP 127.0.0.1:8080 0.0.0.0:0 LISTENING 891  
TCP 127.0.0.1:8080 0.0.0.0:0 LISTENING 13924  
TCP 127.0.0.1:8080 0.0.0.0:0 LISTENING 1920
```

Diagnosis Process

Step 1: Identify the Process

Process Inspection

```
$ ps aux | grep rogue_service.py  
891 python ./scripts/rogue_service.py
```

Identified: The `rogue_service.py` process (PID 891) is occupying port 8080

Root Cause

The incident simulation started a rogue HTTP service deliberately listening on port 8080 with `SO_REUSEADDR` enabled. This simulates a common production incident where:

- A previous service instance crashes but doesn't release the port

- A rogue or compromised process binds to the required port
- The main application cannot start due to "Address already in use" error

Fix Applied

Step 1: Terminate the Rogue Process

```
$ kill 891
```

OR

```
$ kill $(cat scripts/rogue.pid)
```

Step 2: Verify Port is Released

```
$ netstat -ano | grep 8080
```

(No output = port is free)

Service Recovery and Verification

✓ CloudNotes Successfully Started:

```
$ /c/Python314/python ./app/server.py
```

```
[2026-07-09 03:59:20] [INFO] Starting CloudNotes Service...
```

```
[2026-07-09 03:59:20] [INFO] Loading database from .../data/notes.json
```

```
[2026-07-09 03:59:20] [INFO] CloudNotes Service is successfully running  
on http://127.0.0.1:8080
```

Health Check - Final Verification

Health Endpoint Test

```
$ curl -i http://127.0.0.1:8080/health
```

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

```
Content-Length: 20
```

```
{"status": "healthy"}
```

✓ All Verification Passed:

- HTTP Status Code: 200 OK
- Response Body: {"status": "healthy"}
- Service Listening: http://127.0.0.1:8080

Summary of All Fixes

Scenario	Fault Type	Root Cause	Fix Applied	Evidence of Success
1: Configuration	Startup Failure	Wrong JSON key: "db_file_path" vs "database_file_path"	Updated configuration key using sed	Application progresses to next initialization phase
2: Permissions	File Access Denied	Log file chmod 444 (read-only)	Changed to chmod 644 (rw-r--r--)	Log file now writable by owner, application can initialize
3: Port Conflict	Bind Failure	Rogue process listening on port 8080	Killed rogue process (PID 891)	Service binds to port 8080, health check returns 200 OK

Key Learning Outcomes

Troubleshooting Skills Demonstrated

1. **Configuration Management:** Identified and fixed JSON key mismatches that prevent application initialization
2. **File System Administration:** Used chmod to adjust permissions following principle of least privilege (644 instead of 777)

3. **Network Diagnostics:** Used netstat to identify processes bound to specific ports
4. **Process Management:** Killed conflicting processes to free up required resources
5. **Verification:** Implemented health checks to verify service operational status

Recommended Best Practices

- **Configuration:** Use JSON schema validation to catch key mismatches before deployment
- **Permissions:** Set appropriate file permissions during build/deployment (644 for files, 755 for directories)
- **Port Management:** Document required ports and check availability before deployment
- **Monitoring:** Implement health checks and alerting for critical services
- **Documentation:** Maintain runbooks for common incidents (port conflicts, permission errors)

CloudNotes Service Troubleshooting Lab - Evidence Report

Generated: July 9, 2026

Status: All three scenarios successfully diagnosed and remediated ✓